SOFTWARE PROJECT BLACKBOOK

Smart Budget Planner

A full-stack web application for creating monthly budgets, adding daily expenses, verifying users with email OTP, and viewing financial dashboards in real time.

STUDENT NAME
Prathmesh Shinde

DOMAIN
prathmeshshinde.com

PROJECT TYPE
Full-Stack Web Application

DATE
May 2026

Built with: React, Tailwind CSS, Node.js, Express, PostgreSQL, JWT, Resend, Vercel, Render, Neon

📘 Table of Contents

1. Introduction
2. Project Overview
3. Technology Stack
4. System Architecture
5. Modules Explanation
6. APIs Development
7. Database
8. Authentication & Security
9. Docker & Deployment
10. Domain & Hosting
11. Environment Setup
12. Project Workflow
13. Diagrams & Visuals
14. Conclusion

1. Introduction

> Smart Budget Planner is a web application that helps a user plan monthly income, create category-wise budgets, record expenses, and understand where money is being spent.

What the Project Is About

This project converts the idea of an Excel budget tracker into a proper website. In Excel, a user can enter planned budgets and daily spending manually. In this web app, the same logic is made easier through forms, dashboards, charts, filters, authentication, and a database.

Problem Statement

Many people track money in notebooks or Excel files. This works for simple use, but it becomes difficult when the user wants login security, mobile access, monthly history, automatic totals, charts, and safe storage. The problem is to build a digital system where each user can securely manage personal budget data.

Goal of the Project

The goal is to build a production-ready budget planner that can run locally during development and later go live on a real domain using free hosting services.

2. Project Overview

The application has four main parts: a user interface, a backend API, a PostgreSQL database, and an email service for OTP verification.

What the Application Does

- Allows new users to create an account.
- Verifies email using OTP before login.
- Lets users enter monthly income.
- Lets users create categories like Food, Travel, Bills, Savings, or NPF Fund.
- Stores planned budget and actual expenses.
- Shows dashboard cards, pie chart, bar chart, and category comparison.
- Supports edit, delete, monthly filtering, and Excel export.

Key Features

Email + Password Login | **Email OTP** | **User Data Isolation** | **Monthly Budget** | **Expense CRUD** | **Charts** | **Admin View** | **Excel Export** | **Mobile Responsive**

High-Level Explanation

When a user opens the website, React displays the screens. When the user logs in or adds an expense, React sends a request to the Express backend. The backend checks security, validates data, talks to PostgreSQL, and returns the result. The frontend then updates the page without reloading.

User
Uses browser or mobile

→

React Frontend
Forms, dashboard, charts

→

Express API
Business logic and security

→

PostgreSQL
Stores users, budgets, expenses

3. Technology Stack

A technology stack is the group of languages, tools, frameworks, and services used to build and run the project.

Area	Technology	Simple Explanation	Why Used
Language	JavaScript	Main programming language for browser and server.	One language can be used for both frontend and backend.
Frontend	React	Builds pages using reusable components.	Fast UI updates without page reload.
Styling	Tailwind CSS	Utility classes for clean design.	Quick responsive design with consistent colors.
Charts	Recharts	React chart library.	Easy pie chart and bar chart creation.
Backend	Node.js + Express	Runs JavaScript APIs on the server.	Simple, fast REST API development.
Database	PostgreSQL	Relational database with tables.	Reliable financial data storage with relations.
Auth	JWT + bcrypt	Token login and password hashing.	Keeps login secure and avoids storing plain passwords.
Email	Resend	Email sending service.	Sends OTP emails for verification and reset password.
Hosting	Vercel, Render, Neon	Free hosting stack.	Frontend, backend, and database can go live separately.

Programming Languages Used

JavaScript
Used for React frontend and Express backend logic.

SQL
Used to create tables and query PostgreSQL data.

HTML/CSS
Used by the browser to display screens and styling.

Important Tools

GitHub stores source code, Vercel hosts the frontend, Render hosts the backend, Neon hosts PostgreSQL, Resend sends OTP emails, and Docker can run a local PostgreSQL database for testing.

4. System Architecture

System architecture means how different parts of the project are arranged and how they communicate.

Frontend
React app on Vercel
Login, forms, dashboard

→

Backend
Express API on Render
Routes, validation, auth

→

Database
Neon PostgreSQL
Users, categories, budgets

Email OTP
User requests OTP

→

API Service
Generates OTP and hash

→

Resend
Sends email to user

How the Parts Connect

1. The browser loads the React frontend.
2. React uses Axios to call backend API endpoints.
3. The backend checks JWT token for protected routes.
4. The backend reads or writes data in PostgreSQL.
5. The result is sent back as JSON.
6. React updates the dashboard instantly.

Example: When the user adds an expense of ₹250 for Food, the frontend calls `POST /expense`. The backend saves it in the `expenses` table. Then the dashboard total spent and category actual amount are recalculated.

5. Modules Explanation

🔐 Authentication Module

What it does: Handles register, login, email verification, forgot password, reset password, profile update, and password update.

How it works: During signup, the password is hashed with bcrypt and saved. An OTP is emailed to the user. After OTP verification, the user can log in and receives a JWT token.

🔌 API Module

What it does: Provides REST endpoints such as `/dashboard`, `/categories`, `/expense`, and `/income`.

How it works: Each route receives a request, validates input, checks authentication, performs database queries, and returns JSON.

🐘 Database Module

What it does: Stores user accounts, OTP records, income, categories, monthly budgets, and expenses.

How it works: Tables are connected with foreign keys. Every financial table stores a `user_id`, so one user's data does not mix with another user's data.

🎨 Frontend Module

What it does: Shows login pages, dashboard, expense page, category page, settings page, and admin page.

How it works: React components store form values in state. After API success, the component refreshes data and shows a top-center toast message.

📧 Email OTP System

What it does: Sends one-time passwords for email verification and password reset.

How it works: The server creates a short OTP, stores only its hash, sends the OTP through Resend, and verifies it before marking email verified or resetting password.

6. APIs Development

API means Application Programming Interface. In this project, APIs are URL endpoints used by the frontend to communicate with the backend.

Method	Endpoint	Purpose	Protected?
POST	/auth/register	Create user and send verification OTP	No
POST	/auth/login	Login and receive JWT token	No
POST	/auth/verify-email	Verify signup OTP	No
POST	/auth/forgot-password	Send password reset OTP	No
POST	/auth/reset-password	Set new password after OTP	No
GET	/dashboard	Get budget summary and charts data	Yes
GET / POST	/categories	List or create categories	Yes
PUT / DELETE	/categories/:id	Edit or delete category	Yes
POST	/expense	Add daily expense	Yes
GET	/expenses	List monthly expenses	Yes
PUT / DELETE	/expense/:id	Edit or delete expense	Yes
GET	/admin/users	Admin can view users	Admin only

Example API Request

```
POST /expense
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "categoryId": "category-uuid",
  "amount": 250,
  "paymentMethod": "UPI",
  "description": "Lunch",
  "date": "2026-05-02"
}
```

Example API Response

```
{
  "expense": {
```

```
    "id": "expense-uuid",
    "categoryId": "category-uuid",
    "amount": 250,
    "paymentMethod": "UPI",
    "description": "Lunch",
    "date": "2026-05-02"
  }
}
```

7. Database

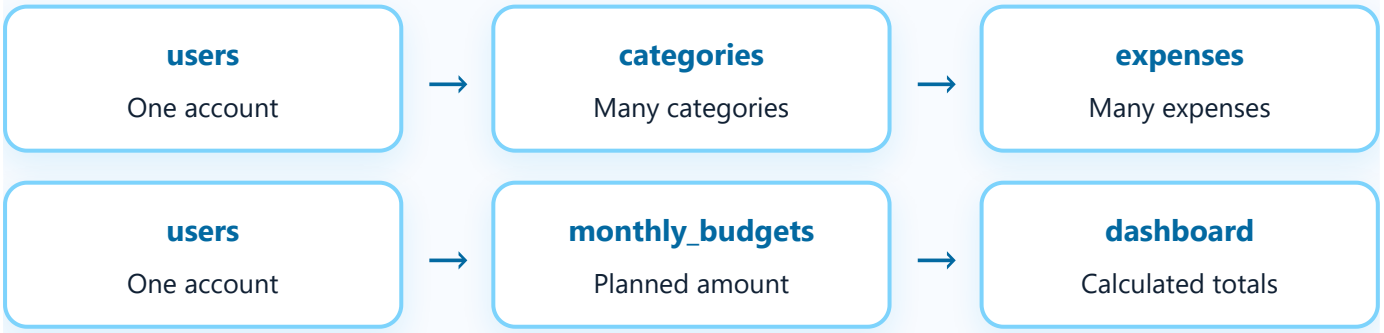
The project uses PostgreSQL, hosted on Neon in production. PostgreSQL is a strong relational database, which is suitable for financial records because relationships and data rules are important.

Why PostgreSQL Was Chosen

- It stores data in structured tables.
- It supports relations using foreign keys.
- It can calculate totals using SQL aggregation.
- It is reliable for money-related records.
- Neon provides a free hosted PostgreSQL option.

Table	Main Purpose	Important Columns
users	Stores account details	id, email, password_hash, name, phone, city, email_verified, role
email_otps	Stores OTP hashes	user_id, purpose, otp_hash, expires_at, consumed_at
categories	Stores budget categories	user_id, name, group_name, default_budget
monthly_budgets	Stores planned amount per category per month	user_id, category_id, month, planned_amount
incomes	Stores monthly income/budget	user_id, month, source, planned_amount, actual_amount
expenses	Stores daily spending	user_id, category_id, amount, payment_method, expense_date

Basic Relationship Diagram



Smart Budget Planner Blackbook | Database

8. Authentication & Security

Login and Signup System

Users sign up with name, email, phone country code, phone number, city, and password. After signup, the user must verify email with OTP. Login is allowed only after email verification.

Email OTP Verification

1. User enters email during signup or forgot password.
2. Backend generates an OTP.
3. Backend stores only a hashed version of the OTP.
4. Resend sends the OTP email from the verified domain email.
5. User enters OTP in the app.
6. Backend checks the OTP and marks it as consumed.

Password Security

Passwords are never stored directly. The backend uses bcrypt to convert the password into a hash. A hash is like a one-way locked version of the password. Even if someone sees the database, they cannot read the original password.

JWT Token

After login, the server gives a signed JWT token. The frontend sends this token with protected requests. The backend checks the token to know which user is making the request.

Environment Variables

Secrets like database URL, JWT secret, and Resend API key are stored in environment variables. They are not written in the source code.

> **Important:** Real passwords, real API keys, and real database URLs should never be included in a report, GitHub repository, or screenshot.

9. Docker & Deployment

What Docker Is

Docker is a tool that packages software into containers. A container is like a small ready-made environment that includes everything needed to run a service. For example, PostgreSQL can run inside a Docker container without installing PostgreSQL directly on the computer.

How Docker Was Used

Docker was used for local PostgreSQL testing through `docker-compose.yml`. This lets the developer test with a real PostgreSQL database before using the hosted Neon database.

```
docker compose up -d postgres
```

Container Setup Overview

Developer PC
Runs commands locally

→

Docker Compose
Starts PostgreSQL service

→

PostgreSQL Container
Local database for testing

Production Deployment

For the live website, separate services were used:

- **Vercel:** hosts the React frontend.
- **Render:** hosts the Express backend API.
- **Neon:** hosts the PostgreSQL database.
- **GitHub:** stores source code and triggers deployments.

10. Domain & Hosting

Domain Purchased via GoDaddy

The project domain is **prathmeshshinde.com**. A domain is the human-readable website name. Instead of remembering a technical hosting URL, users can open the project through a custom domain or subdomain.

How the Domain Is Connected

1. Buy or manage the domain in GoDaddy.
2. Create frontend project in Vercel.
3. Add the custom domain or subdomain in Vercel project settings.
4. Copy DNS records shown by Vercel.
5. Paste those records in GoDaddy DNS management.
6. Wait for DNS propagation.
7. Open the domain and test the website.

Hosting Process

GitHub
Source code

→

Vercel
Frontend build

→

Domain
Public website

GitHub
Source code

→

Render
Backend server

→

Neon
Database

Making the Website Live

After frontend and backend are deployed, the frontend environment variable `VITE_API_URL` points to the Render API URL. The backend `CLIENT_ORIGINS` allows requests from the Vercel/domain URL. This connection makes the full project work live.

11. Environment Setup

Environment variables are settings that change between local testing and production. They help keep source code clean and secrets private.

Backend .env Example

```
NODE_ENV=development
PORT=4001
DATABASE_URL=postgresql://USER:PASSWORD@HOST/DBNAME?sslmode=require
JWT_SECRET=replace-with-a-long-random-secret
CLIENT_ORIGINS=http://localhost:5173
TOKEN_EXPIRES_IN=7d
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxx
EMAIL_FROM="Smart Budget Planner <noreply@example.com>"
```

Frontend .env Example

```
VITE_API_URL=http://localhost:4001
```

Why We Use Placeholders

The examples above show the shape of the configuration only. Real values must be stored privately on the local machine or in hosting dashboards like Render and Vercel.

> **Best Practice:** Keep `.env` files out of GitHub by listing them in `.gitignore`.

12. Project Workflow

Development Process

1. Studied the Excel budget planner structure.
2. Created database schema for users, categories, monthly budgets, income, OTPs, and expenses.
3. Built Express API routes for authentication, dashboard, category, expense, income, export, and admin views.
4. Built React screens for login, register, email verification, forgot password, dashboard, expenses, categories, settings, and admin.
5. Connected frontend and backend using Axios.
6. Added email OTP using Resend.
7. Tested locally with localhost.
8. Connected hosted PostgreSQL database using Neon.
9. Deployed backend on Render and frontend on Vercel.
10. Prepared custom domain connection through GoDaddy and Vercel.

How Everything Connects Together

Input	→	Process	→	Output
User enters income, category, expense		API validates and saves data		Dashboard updates totals and charts

Important Budget Logic

- **Total Budget:** Monthly income/budget entered by user.
- **Total Planned:** Sum of all category planned budgets.
- **Total Spent:** Sum of all expenses for selected month.
- **Remaining:** Total Budget minus Total Spent.
- **Category Actual:** Sum of expenses for that category in selected month.
- **Warning:** Show alert only when spending crosses planned category budget.

13. Diagrams & Visuals

Architecture Diagram

🌐 Browser
React pages
Dashboard UI

⇄

🧠 REST API
Express routes
JWT auth
Zod validation

⇄

🗄 PostgreSQL
Tables
Relations
Aggregations

Data Flow Diagram

1. User Action
Add expense ₹500

→

2. API Request
POST /expense

→

3. Database
Insert row in expenses

4. Dashboard API
GET /dashboard

→

5. Calculations
Total spent, remaining

→

6. UI Update
Charts and cards refresh

Module Connection Diagram

Auth Module
Protects user account and creates JWT.

Budget Module
Stores income and planned category budgets.

Expense Module
Records spending by category and date.

Dashboard Module
Aggregates totals and chart data.

Email Module
Sends OTP for verification and reset.

Admin Module
Allows read-only user review and user delete.

14. Conclusion

What Was Achieved

Smart Budget Planner was successfully built as a full-stack web application. It supports real user accounts, email OTP verification, PostgreSQL database storage, monthly budgeting, category planning, daily expense logging, dashboard charts, Excel export, and deployment-ready configuration.

What Was Learned

- How to convert Excel logic into a real web application.
- How frontend, backend, database, and APIs connect.
- How to store financial data securely per user.
- How password hashing and JWT authentication work.
- How email OTP improves account verification.
- How to deploy a project using Vercel, Render, Neon, and a GoDaddy domain.

Future Improvements

- Add stronger admin controls with audit logs.
- Add recurring expenses like rent or subscriptions.
- Add email alerts when a category crosses planned budget.
- Add PDF reports for monthly summary.
- Add bank statement import with automatic categorization.
- Add two-factor authentication for admin users.

Final Result: The project is simple enough for daily use, but technically strong enough to be hosted as a real website with separate frontend, backend, and database services.

Prepared for Smart Budget Planner | Built by Prathmesh Shinde | prathmeshshinde.com